FakeShield — AI Image Lab: Full Architecture Report

> ***Engine Version:*** *FakeShield-v8.0-MultiSignal*
> ***Backend Port:*** *8001* · ***API Route:*** `POST /api/v1/image/analyze`
> ***Frontend:*** *React + Framer Motion (* `ImageLabPage.tsx` *)*

---

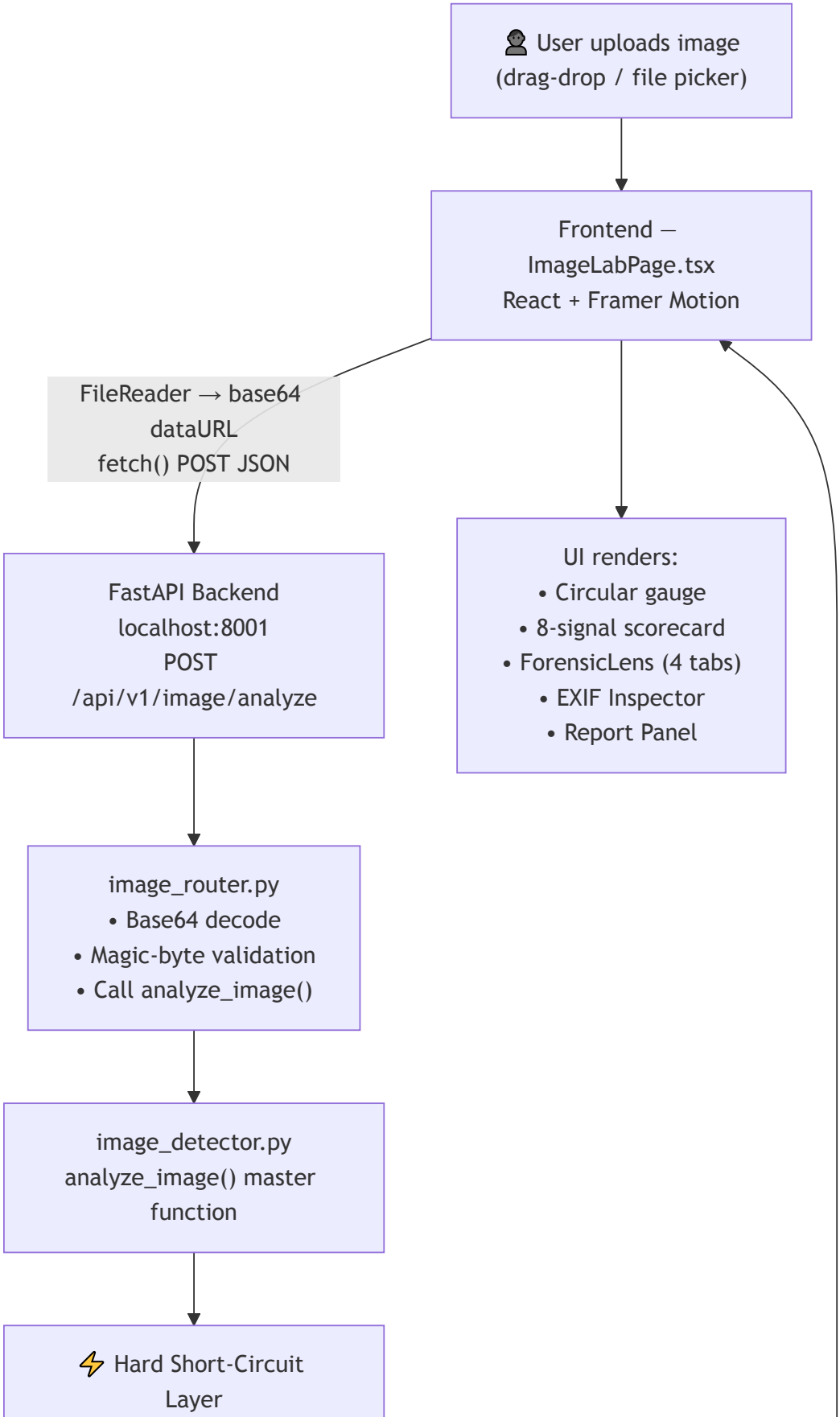
1. What is the AI Image Lab?

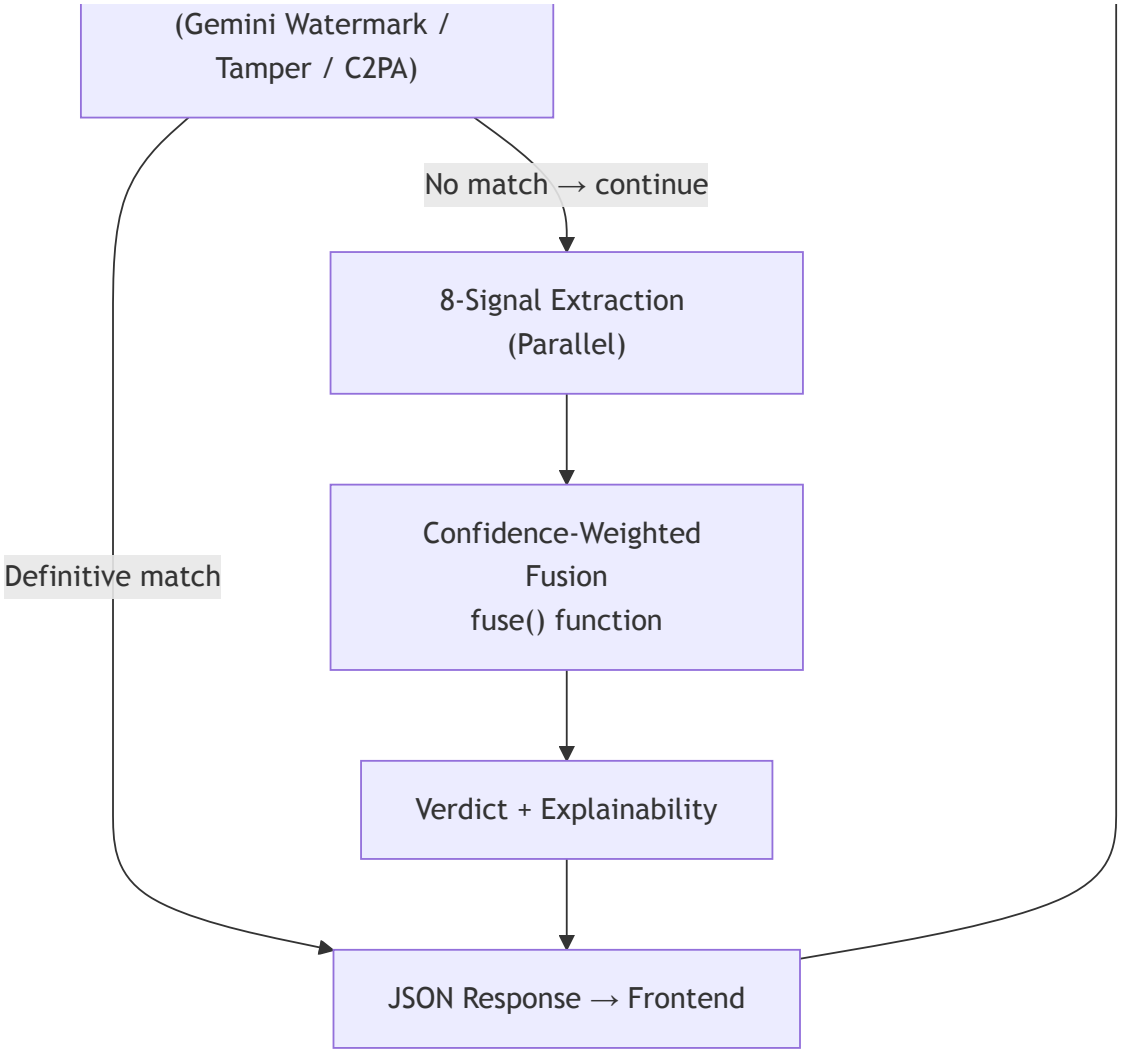
The AI Image Lab is FakeShield's **multimodal forensic engine** for detecting whether a photograph or digital image was created by an AI generator (Stable Diffusion, Midjourney, DALL-E, FLUX, etc.) or taken by a real camera.

It runs **8 independent forensic signals** in parallel, each targeting a different fingerprint left by AI generators, then fuses all scores into a single verdict using **confidence-weighted fusion**.

---

2. High-Level System Architecture


User uploads image
(drag-drop / file picker)
Frontend —
ImageLabPage.tsx
React + Framer Motion
FileReader → base64
dataURL
fetch() POST JSON
FastAPI Backend
localhost:8001
POST
/api/v1/image/analyze
UI renders:
• Circular gauge
• 8-signal scorecard
• ForensicLens (4 tabs)
• EXIF Inspector
• Report Panel
image_router.py
• Base64 decode
• Magic-byte validation
• Call analyze_image()
image_detector.py
analyze_image() master
function
Hard Short-Circuit
Layer



---

3. Frontend Data Flow

Input

- User selects or drags an image (JPG / PNG / WEBP / BMP / GIF / TIFF, max 20 MB)
- `processFile()` reads it twice:
  - `FileReader.readAsDataURL` → browser preview (instant)
  - Same `dataURL` sent as JSON body to backend

HTTP Request

```
POST http://localhost:8001/api/v1/image/analyze
{
  "image": "...",
  "include_gradcam": true
}
```

Output Rendered by Frontend

UI Component	Data from Backend
Circular Gauge	`ai_probability` (0–1)
Verdict Badge	`verdict` + `threat_level`
8-Signal Scorecard	`signals{}` object
ForensicLens Tab 1	Original image (browser preview)
ForensicLens Tab 2	`fft_spectrum_url` (base64 PNG)
ForensicLens Tab 3	`heatmap_url` (base64 PNG)
ForensicLens Tab 4	`ela_image` (base64 PNG)
EXIF Inspector	`metadata{}` — camera/gps/lens/software
Report Panel	`reasons[]` + `per_generator_accuracy{}`

---

4. Backend Entry & Validation

`image_router.py`

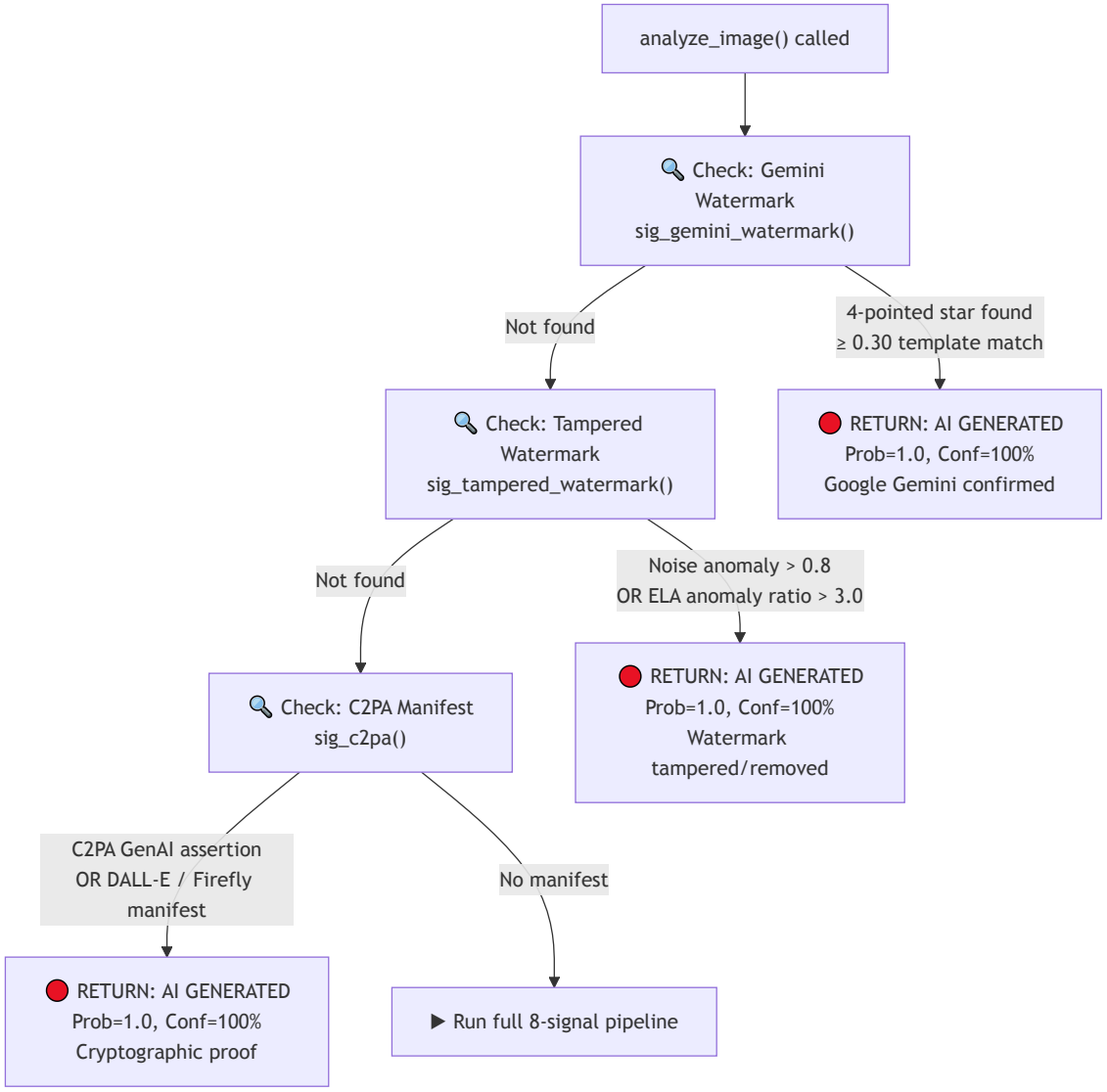
Input: Raw HTTP POST with base64 image string
Steps:

1. Strip base64 prefix (``)
2. Fix padding, strip whitespace
3. `base64.b64decode()` → raw bytes
4. **Magic-byte validation** — checks file header bytes:
   - JPEG: `FF D8 FF`
   - PNG: `89 50 4E 47`
   - WEBP: `RIFF`
   - BMP: `BM` , TIFF: `II*` , GIF: `GIF`
5. Rejects HTML responses with clear error message
6. Calls `analyze_image(image_bytes, include_gradcam)`

---

5. Hard Short-Circuit Layer

Before any expensive model inference, three instant checks can **short-circuit** the entire pipeline and return `AI GENERATED` with 100% confidence:



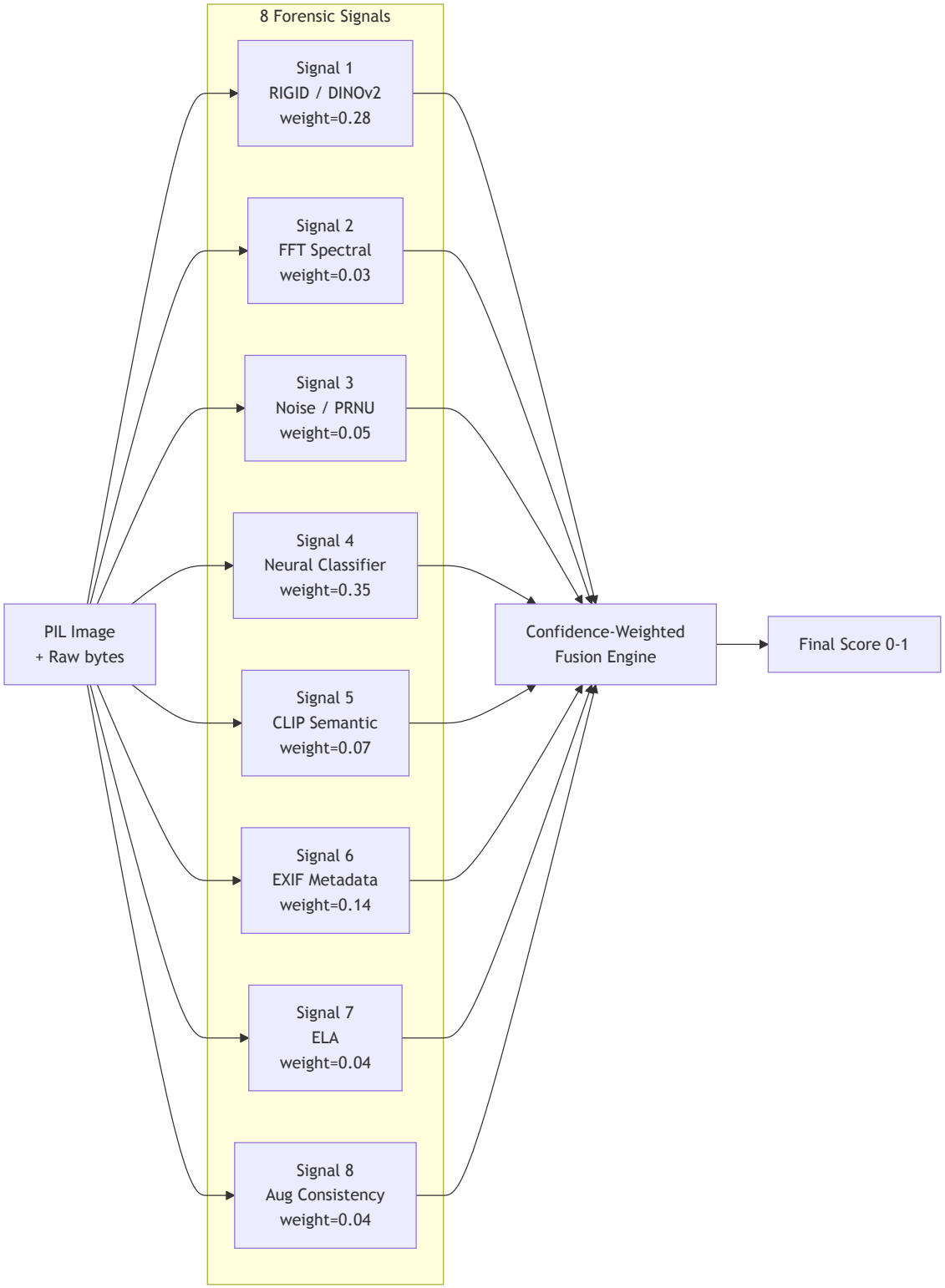
What each short-circuit detects

Check	Method	Target
Gemini Watermark	OpenCV template matching on bottom-right corner	Google Imagen/Gemini 4-pointed star
Tamper Detection	Noise variance anomaly + ELA comparison	Images where watermark was inpainted/healed
C2PA Manifest	`c2pa-python` SDK parses embedded manifest	DALL-E 3, Adobe Firefly, any CAI-signed image

---

6. The 8 Forensic Signals (Full Pipeline)

When no short-circuit fires, all 8 signals run sequentially:

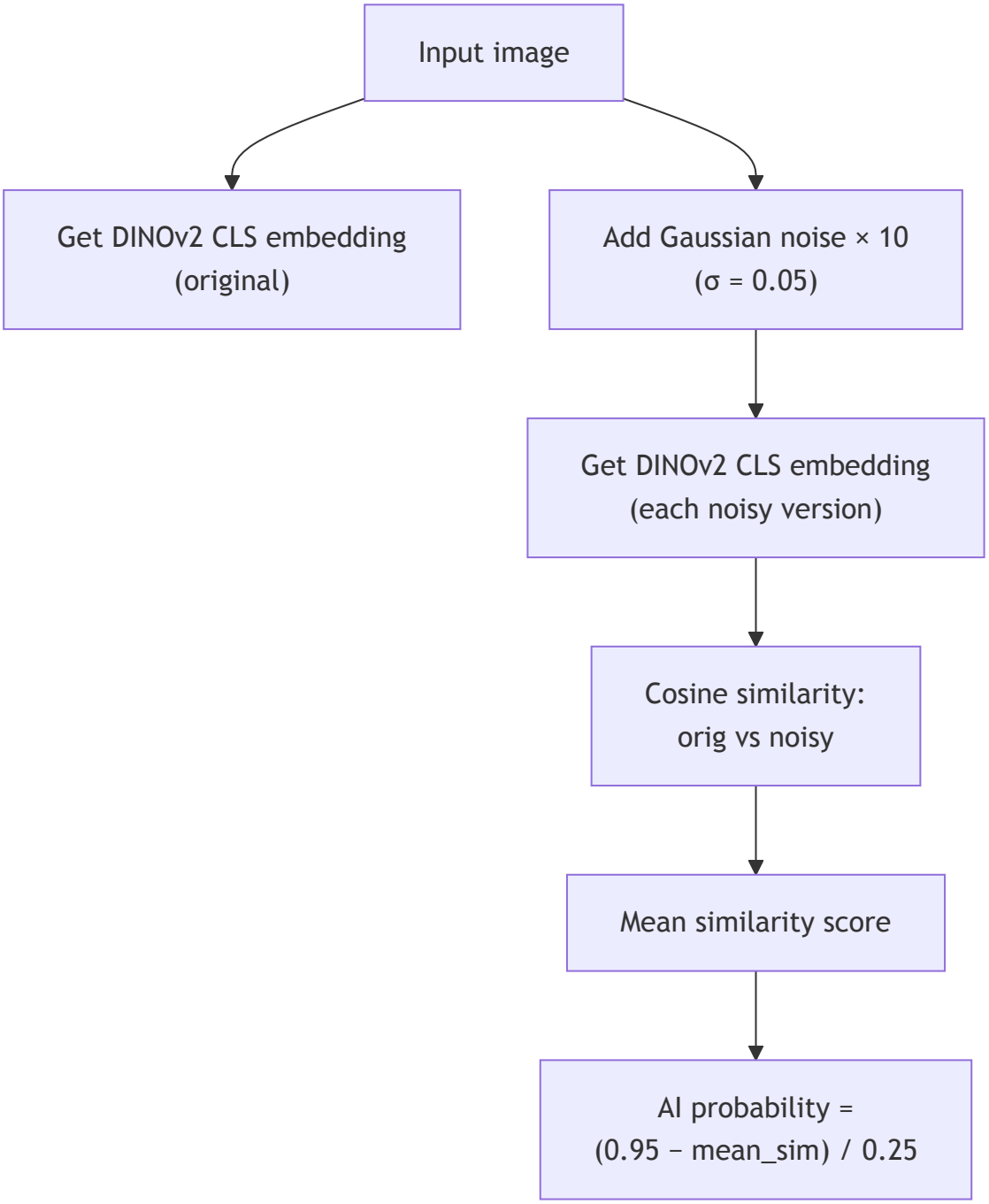


---

7. Signal-by-Signal Deep Dive

Signal 1 — RIGID / DINOv2 (Weight: 0.28)

Model: `facebook/dinov2-base`
Method: Training-free perturbation sensitivity test



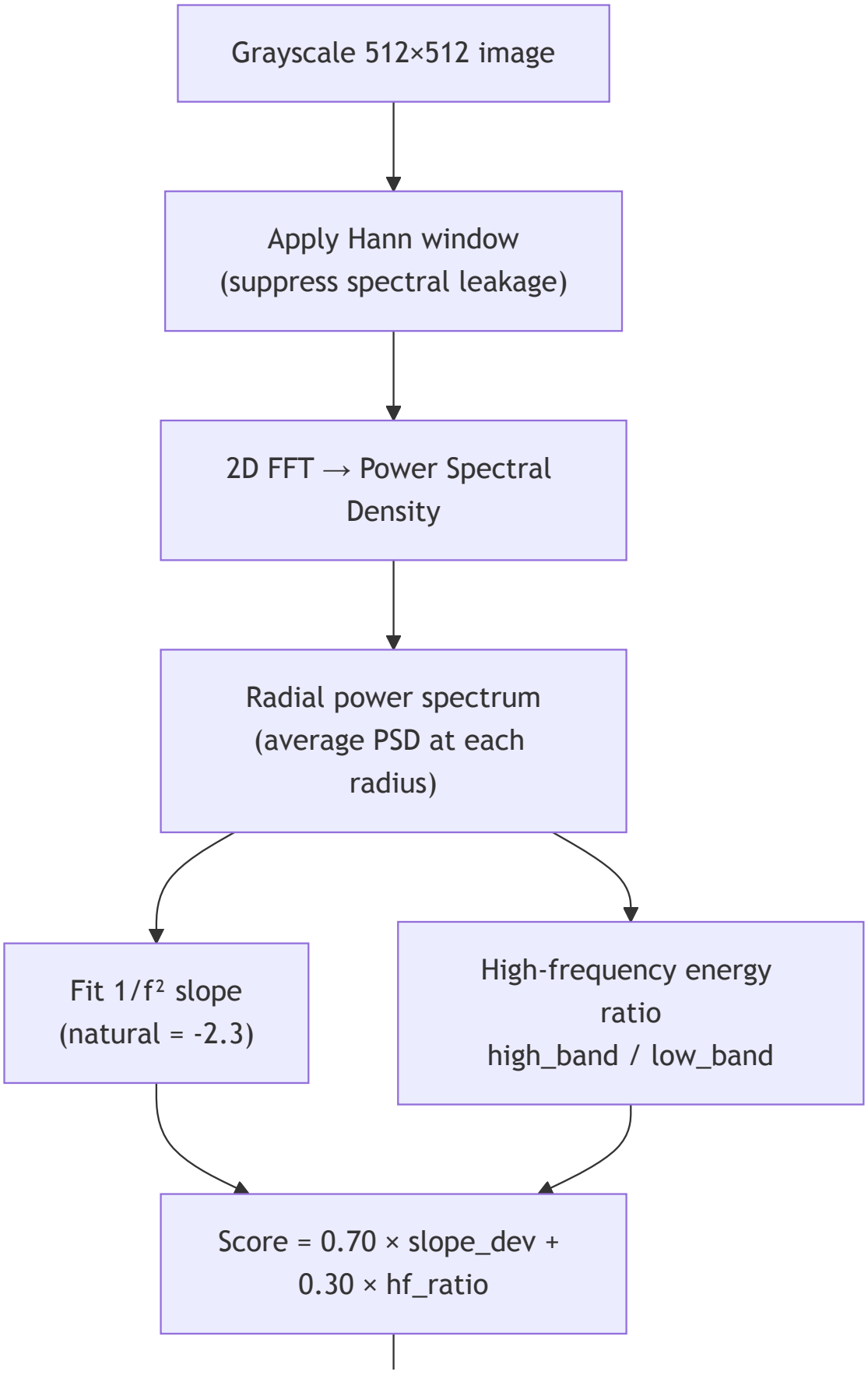
Key insight:

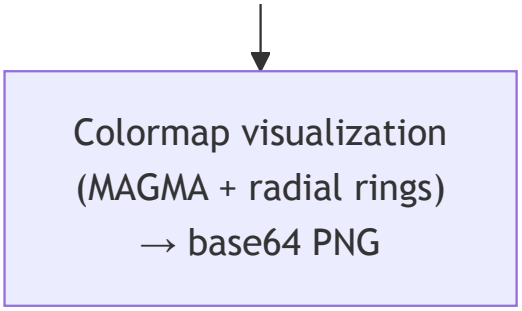
- **Real photos:** stable under noise → HIGH cosine similarity → LOW AI score
- **AI images:** sensitive to noise → LOW similarity → HIGH AI score

Output: `(ai_prob: float, confidence: float)`

Signal 2 — FFT Spectral Analysis (Weight: 0.03)

Method: Radial Integral Operation (RIO) on 2D FFT power spectrum


Grayscale 512×512 image
Apply Hann window
(suppress spectral leakage)
2D FFT → Power Spectral
Density
Radial power spectrum
(average PSD at each
radius)
Fit 1/f² slope
(natural = -2.3)
High-frequency energy
ratio
high_band / low_band
Score = 0.70 × slope_dev +
0.30 × hf_ratio



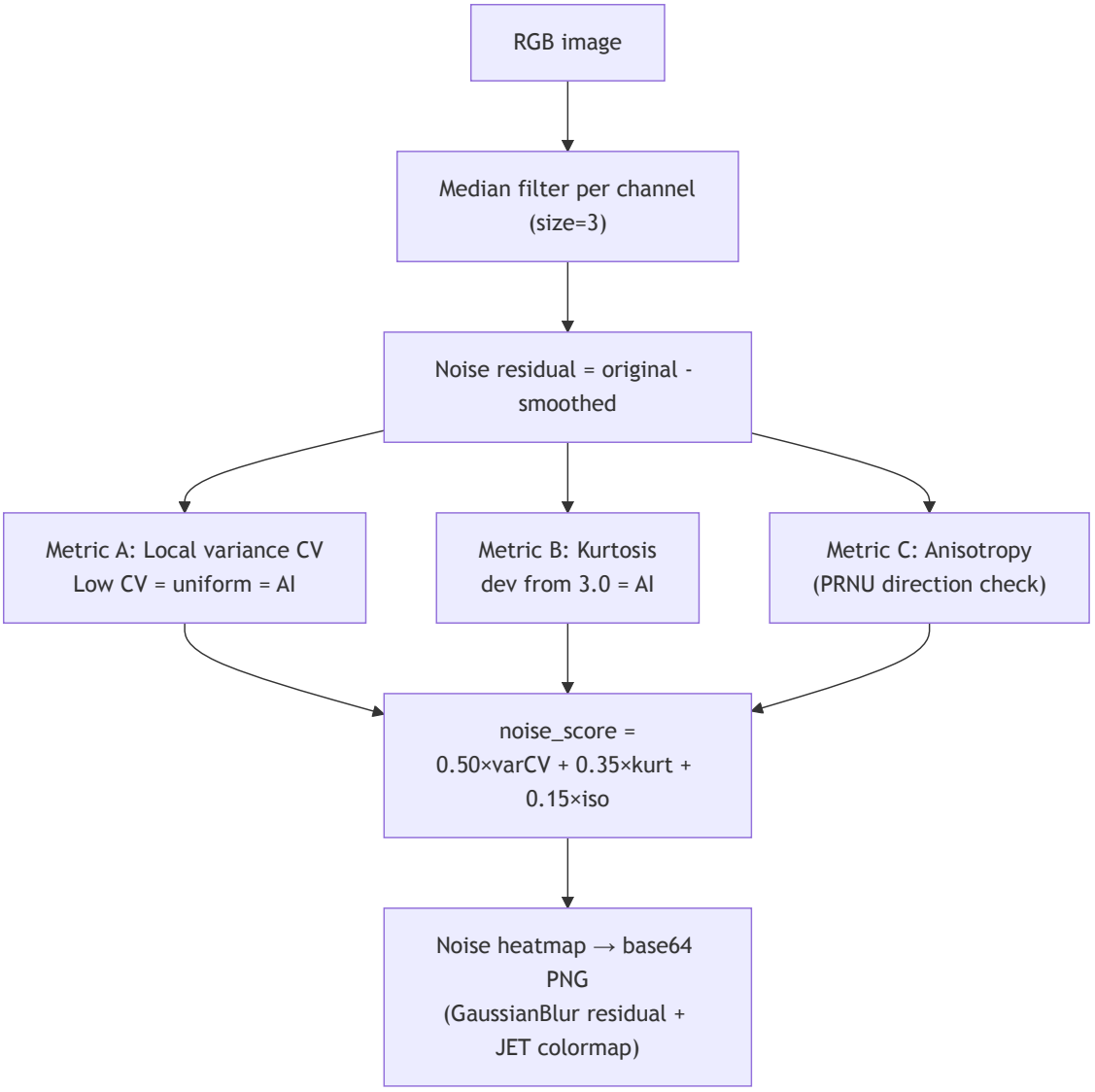
Key insight:

- Natural photos follow $1/f^2$ power decay (slope ≈ −2.3)
- AI images: flat high-freq plateau OR periodic upsampling spikes

Output: `(score, confidence, fft_vis_b64)`
Visual sent to frontend: FFT Spectrum tab in ForensicLens

Signal 3 — Noise Pattern / PRNU (Weight: 0.05)

Method: SRM-like high-pass residual analysis

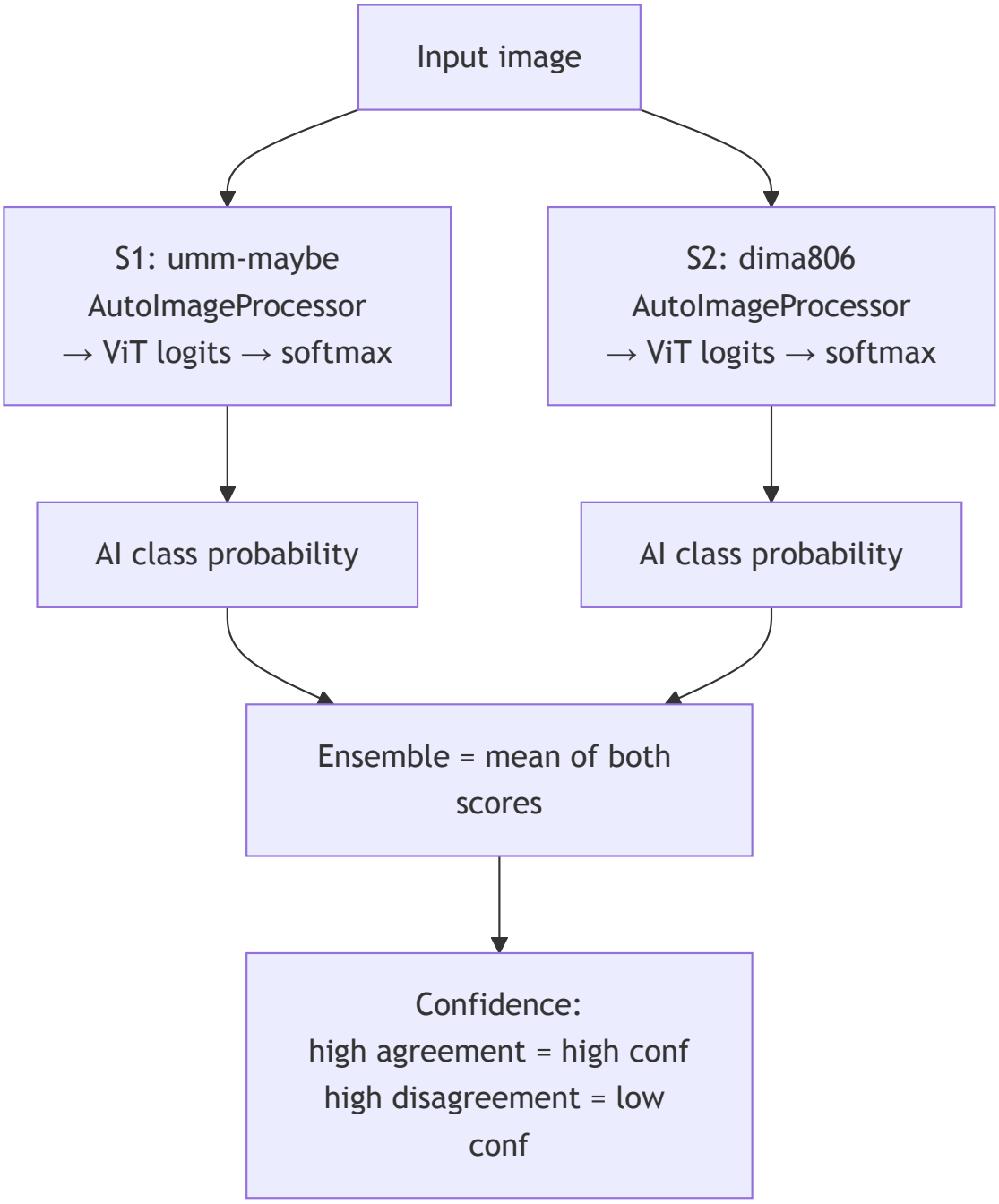


Output: `(score, confidence)` + heatmap for Noise Map tab

Signal 4 — Neural Classifier Ensemble (Weight: 0.35)

Models:

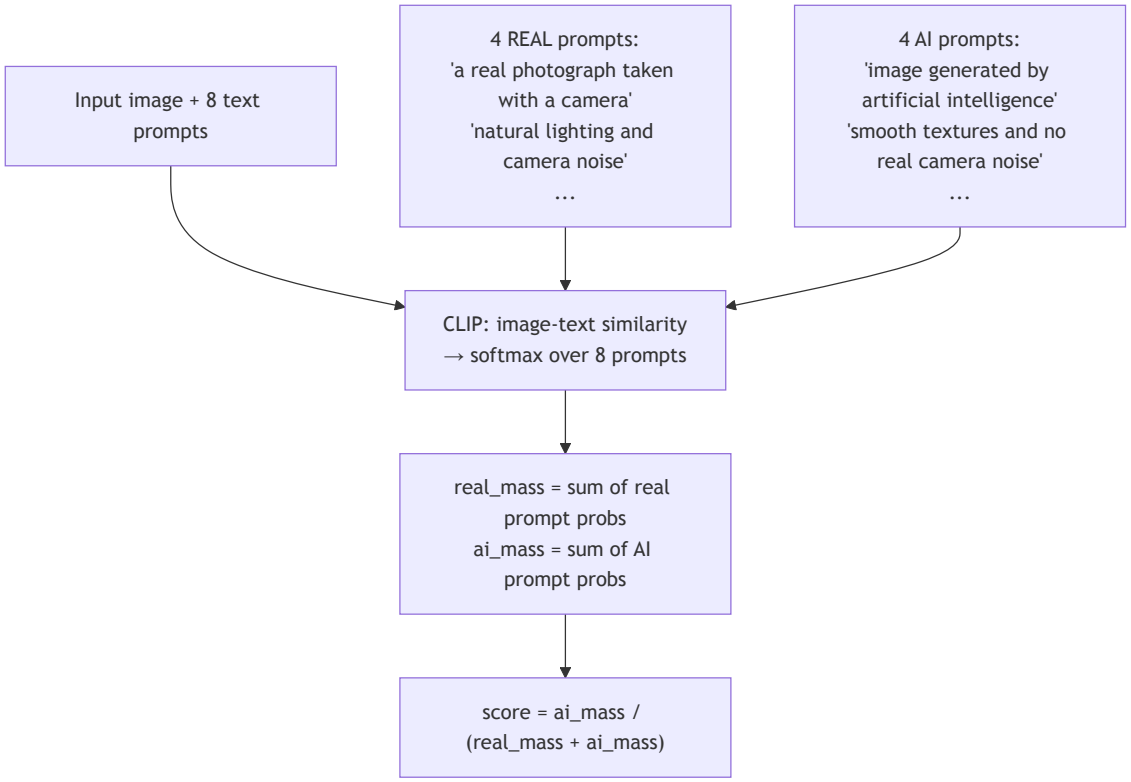
1. `umm-maybe/AI-image-detector` (ViT-based)
2. `dima806/deepfake_vs_real_image_detection` (ViT-based)



Input: PIL image → processor → PyTorch tensors
Output: `(ensemble_score, confidence)` — highest weight signal

Signal 5 — CLIP Semantic (Weight: 0.07)

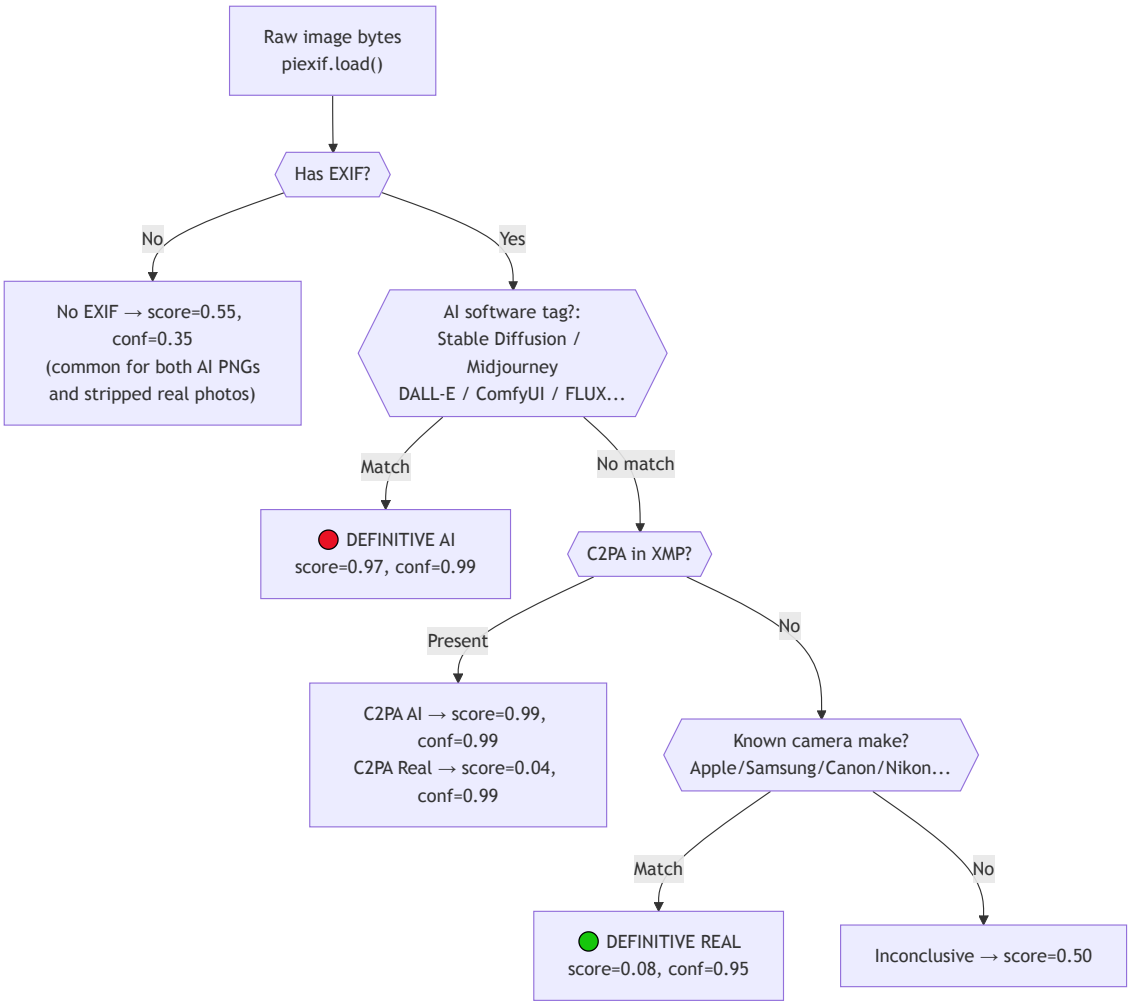
Model: `openai/clip-vit-large-patch14` (falls back to `clip-vit-base-patch32`)
Method: Zero-shot contrastive classification with engineered prompt pairs



Output: `(score, confidence)`

Signal 6 — EXIF Metadata Guard (Weight: 0.14)

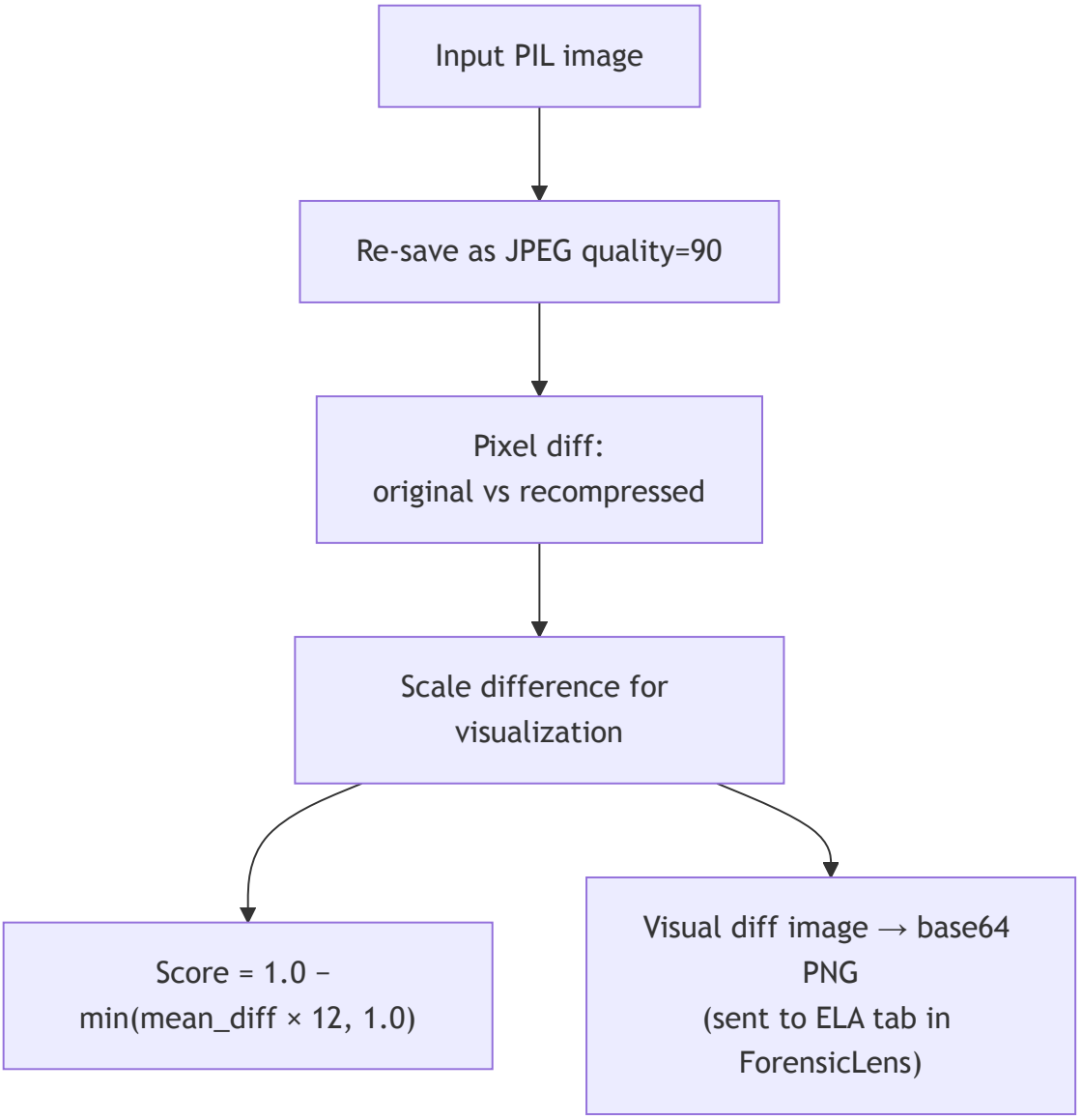
Method: Rule-based binary decision tree on EXIF tags



Hard Veto: If EXIF confidence ≥ 0.90, EXIF score overrides all other signals entirely.

Signal 7 — ELA (Error Level Analysis) (Weight: 0.04)

Module: `image_ela.py`

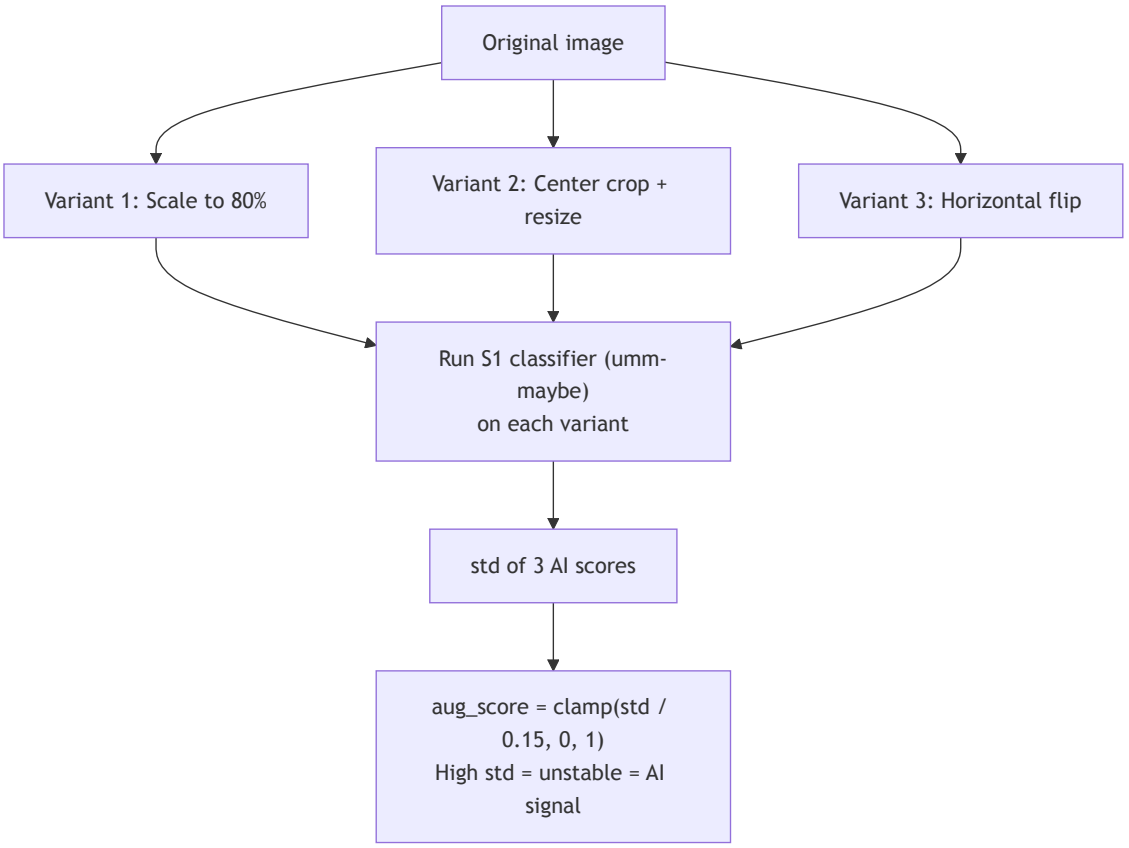


Key insight:

- AI images: uniform latent generation → very low compression error → high ELA score (AI)
- Real photos: sensor noise + complex textures → uneven high ELA → low ELA score (Real)

Signal 8 — Augmentation Consistency (Weight: 0.04)

Method: Classifier stability test under image transforms



---

8. Fusion Engine

Formula:

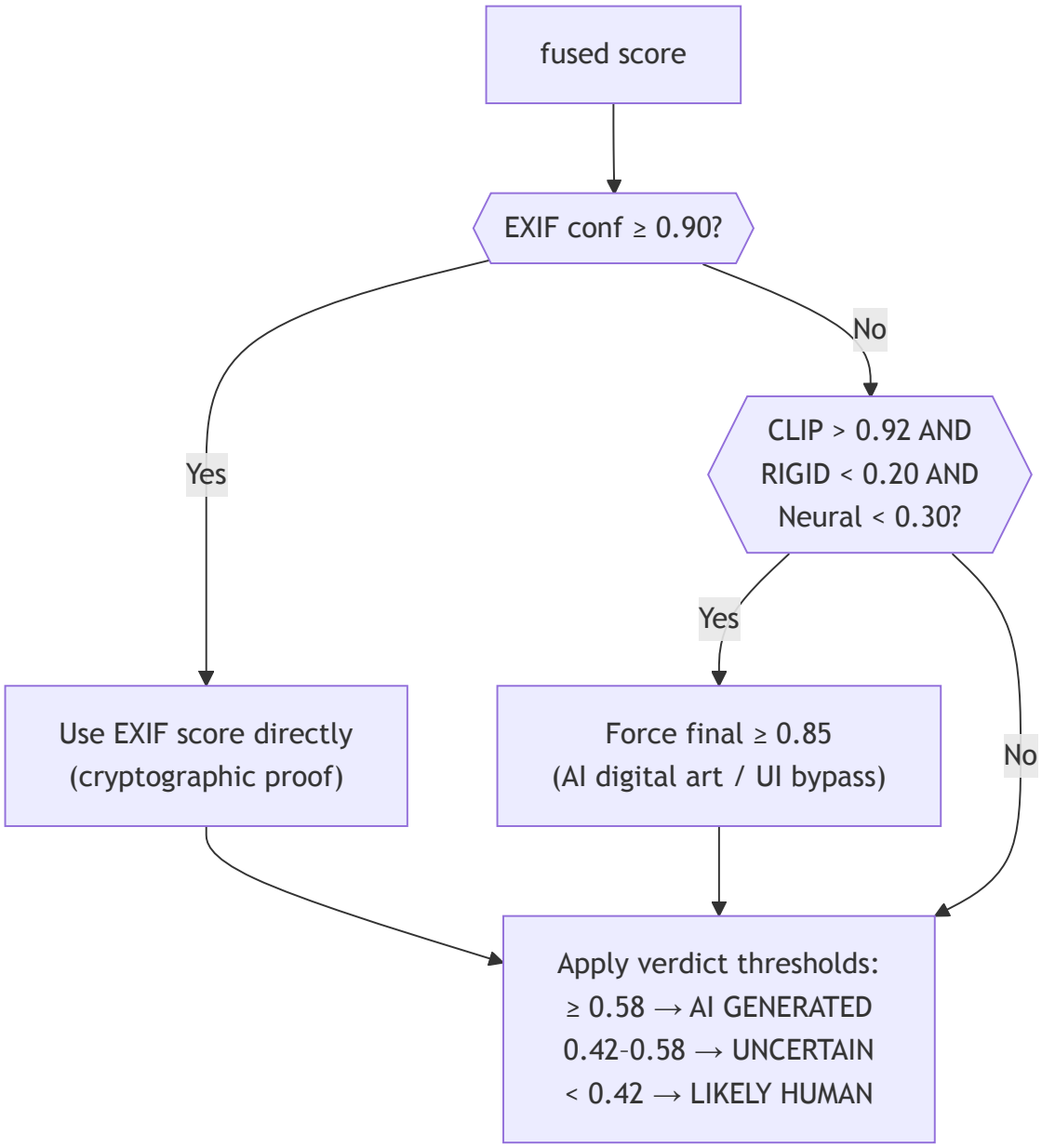
```
effective_weight_i = base_weight_i × confidence_i  (if conf ≥ 0.4)
                   = base_weight_i × confidence_i × 0.5  (if conf < 0.4)

final_score = Σ(effective_weight_i × score_i) / Σ(effective_weight_i)
```

Weight table:

Signal	Base Weight	Notes
Neural Classifier (ViT × 2)	**0.35**	Highest — strongest learnt detector
RIGID / DINOv2	**0.28**	Training-free, best generalization
EXIF Metadata	**0.14**	Hard veto if conf ≥ 0.90
Noise / PRNU	**0.05**	Moderate reliability
CLIP Semantic	**0.07**	Zero-shot domain gap
FFT Spectral	**0.03**	Legacy GAN artifacts
ELA	**0.04**	Compression uniformity

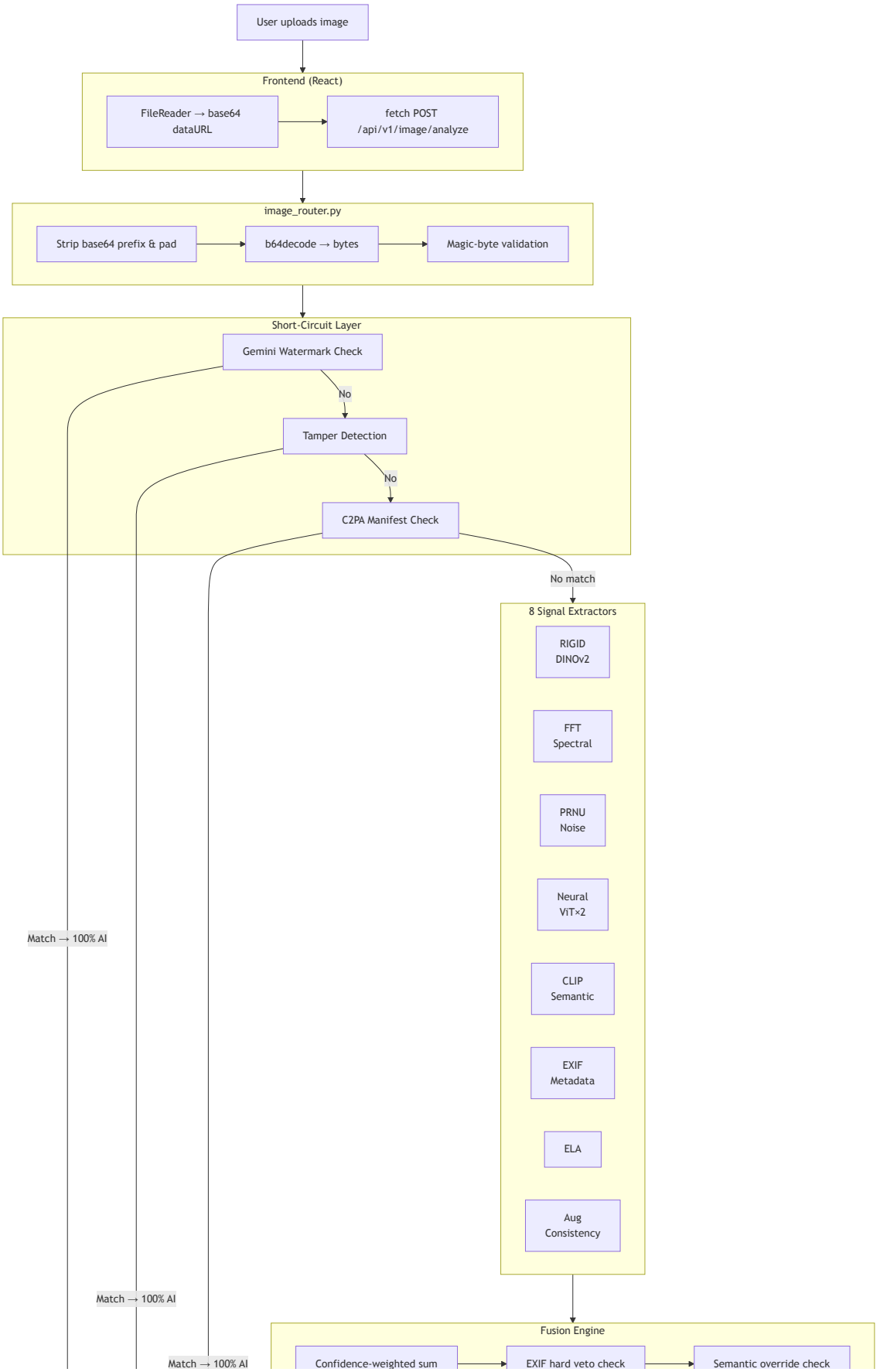
Aug Consistency	**0.04**	Classifier stability

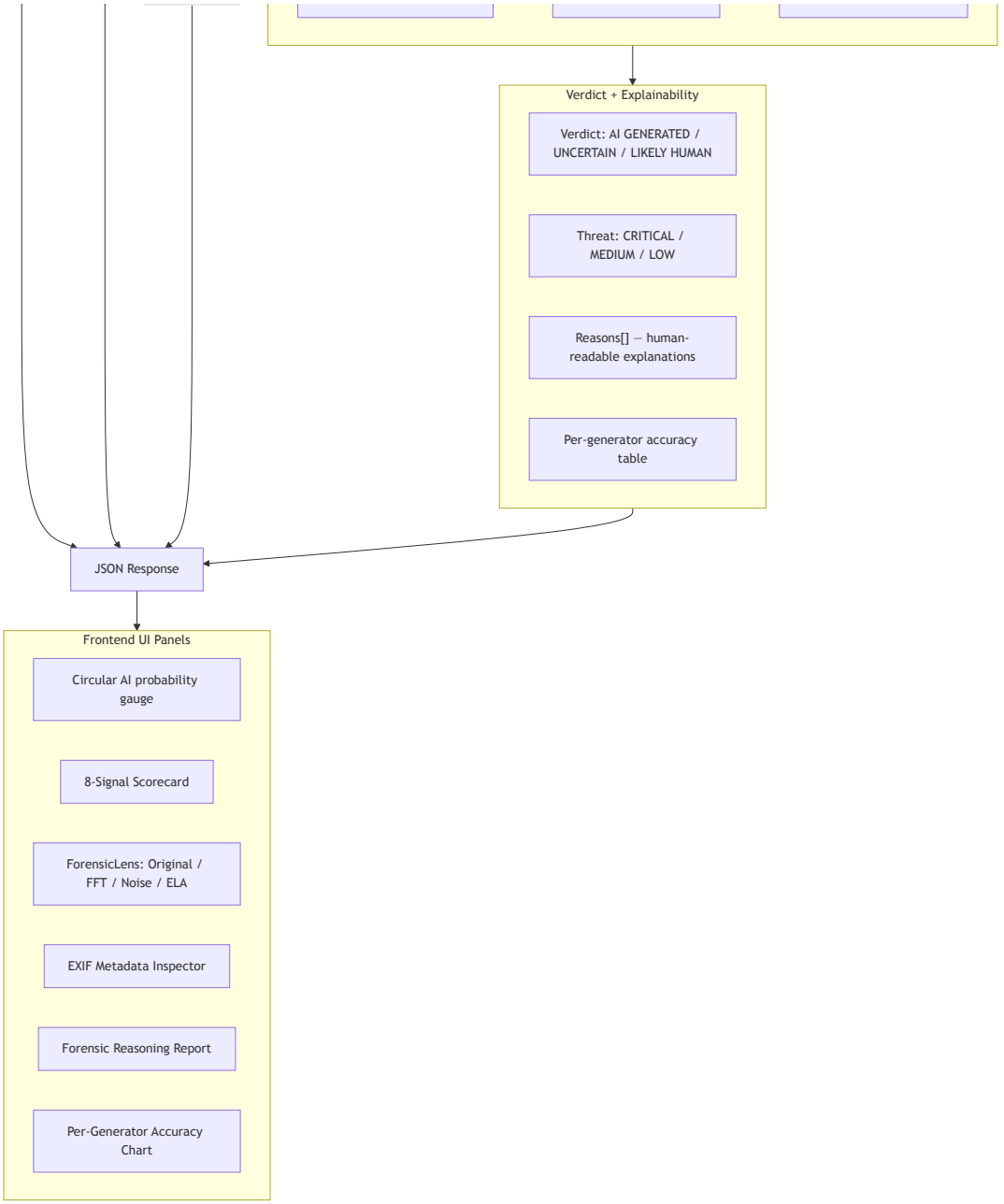
Special overrides after fusion:



---

9. Complete End-to-End Flowchart


User uploads image
Frontend (React)
FileReader → base64 dataURL
fetch POST /api/v1/image/analyze
image_router.py
Strip base64 prefix & pad
b64decode → bytes
Magic-byte validation
Short-Circuit Layer
Gemini Watermark Check
No
Tamper Detection
No
C2PA Manifest Check
No match
8 Signal Extractors
RIGID DINOv2
FFT Spectral
PRNU Noise
Neural ViT×2
CLIP Semantic
EXIF Metadata
ELA
Aug Consistency
Match → 100% AI
Match → 100% AI
Match → 100% AI
Fusion Engine
Confidence-weighted sum
EXIF hard veto check
Semantic override check



---

10. Models Used — Summary Table

Model	Source	Purpose	Framework
DINOv2-base	`facebook/dinov2-base`	RIGID perturbation sensitivity	HuggingFace Transformers
umm-maybe AI detector	`umm-maybe/AI-image-detector`	Neural ViT classifier (primary)	HuggingFace Transformers
dima806 deepfake detector	`dima806/deepfake_vs_real_image_detection`	Neural ViT classifier (backup)	HuggingFace Transformers

CLIP Large	`openai/clip-vit-large-patch14`	Zero-shot semantic classification	HuggingFace Transformers
c2pa-python	PyPI `c2pa` SDK	Content Authenticity Initiative manifests	C2PA SDK
piexif	PyPI	EXIF tag parsing	Python
OpenCV	`cv2`	FFT, noise heatmap, template matching	OpenCV
SciPy	`scipy.signal`, `ndimage`	Noise residual, median filter	SciPy

---

11. Input / Output Contract

Input to `analyze_image()`

```
image_bytes: bytes       # Raw decoded image bytes
include_gradcam: bool    # Whether to generate heatmap/FFT visualizations
```

Output JSON structure

```
{
  "ai_probability": 0.87,          // 0.0 = human, 1.0 = AI
```

```
  "confidence": 76.4,              // 0-100%
  "verdict": "AI GENERATED",       // AI GENERATED | UNCERTAIN | LIKELY HUMAN
  "threat_level": "CRITICAL",      // CRITICAL | MEDIUM | LOW
  "signals": {
    "rigid":      0.72,
    "fft":        0.61,
    "exif":       0.55,
    "classifier": 0.91,
    "clip":       0.84,
    "noise":      0.43,
    "ela":        0.68,
    "aug":        0.55,
    "c2pa":       0.0
  },
  "metadata": {
    "camera": "NONE",
    "gps": "NONE",
    "lens": "NONE",
    "software": "NONE",
    "dimensions": "1024×1024"
  },
  "reasons": ["✗ Neural classifier (91%): ...", "○ No EXIF camera data..."],
  "fft_spectrum_url": "...",
  "heatmap_url": "...",
  "ela_image": "...",
  "per_generator_accuracy": {
    "Midjourney v6-v7": { "accuracy": "~40%", "notes": "..." }
  },
  "processing_time": "4.2s",
  "engine_version": "FakeShield-v8.0-MultiSignal"
}
```

---

12. Per-Generator Detection Accuracy

AI Generator	Accuracy	Primary Signal
DALL-E 3 / ChatGPT	~95%+	C2PA manifest (cryptographic)
Adobe Firefly	~90%+	C2PA manifest
ProGAN / StyleGAN2	~85%	PRNU Noise + FFT
Stable Diffusion 1.4–2.1	~72%	Neural ViT classifiers
SDXL / SD 3.5	~58%	RIGID + ensemble
Google Gemini (Imagen)	100%	Visible watermark
Midjourney v6–v7	~40%	RIGID + EXIF only
FLUX Dev / Schnell	~35%	Very hard — honest score

---

13. Verdict Thresholds

Score Range	Verdict	Threat Level	Frontend Color
≥ 0.58	**AI GENERATED**	CRITICAL	🔴 Red
0.42 – 0.58	**UNCERTAIN**	MEDIUM	🟡 Yellow
< 0.42	**LIKELY HUMAN**	LOW	🟢 Green

---

14. Frontend Components Map

File	Role
`ImageLabPage.tsx`	Main page — upload, state management, layout
`ForensicLens.tsx`	4-tab image viewer (Original / FFT / Noise Map / ELA)
`MetadataInspector.tsx`	EXIF field display with colour-coded badges
`ImageReportPanel.tsx`	Forensic reasons list + per-generator accuracy chart
`ELAViewer.tsx`	Standalone ELA image display with legend
`imageService.ts`	`analyzeImage()` fetch call + TypeScript response interface

Report generated for FakeShield Final Year Project — AI Image Lab v8.0